LogiEdge Final Report

Executive summary

LogiEdge is an offline-first edge intelligence prototype for FreightBridge’s refrigerated fleet. The platform samples temperature and vibration on each truck, extracts six fused features from 30-second windows, and runs a small on-device MLP to classify each window as Normal, Warning, or Critical. Results are written to a local alert log and synchronised after connectivity returns.

This design keeps the system operational during rural cellular outages, reduces uplink bandwidth, and preserves event-chain integrity without sending raw telemetry to the cloud.

Requirement fit

The assignment requires a system that detects cold-chain faults under strict latency, bandwidth, and connectivity constraints. The implemented design satisfies these goals by:

processing sensor streams locally on the truck,

maintaining a local MQTT broker and local alert log,

reducing raw telemetry to compact event records, and

retaining a deterministic inference pipeline that runs without cloud connectivity.

Architecture and constraints

Latency

The operational target is detection and alerting within 90 seconds of a fault signature. Local filtering and inference remove the cloud round trip and the dependence on rural cellular scheduling. This keeps the safety decision local and deterministic.

Bandwidth

Raw telemetry would be expensive: temperature at 1 Hz and vibration at 0.5 Hz generate hundreds of megabytes of data per truck per day. LogiEdge avoids that by sending only compact event records and maintaining the local log as the source of truth during outages.

Connectivity

The prototype is designed for offline operation. During outages, the local MQTT broker and inference service continue to operate, and queued alerts are sent later over the cellular uplink once connectivity is restored.

Privacy and data minimisation

Sensitive temperature and vibration streams remain on the truck. The backend receives minimal alert records, preserving privacy while still enabling operational monitoring and evidence retention.

Data pipeline

The simulator generates deterministic temperature and vibration streams with controlled anomaly modes: normal, temperature drift, vibration fault, and combined drift. The feature extractor computes a six-value fused vector using the project-defined metrics:

1. temperature mean
2. temperature standard deviation
3. temperature rate of change
4. vibration RMS
5. vibration peak magnitude
6. vibration kurtosis

A fixed training-stat normalisation layer is computed from clean data and persisted to `data_pipeline/training_stats.npy`. This keeps the inference pipeline deterministic and portable across runs.

Model and training

The training pipeline creates a labelled dataset from the simulator and train/validation split. The generated dataset contains 294 windows:

118 Normal

88 Warning

88 Critical

The NumPy MLP uses two hidden layers with ReLU activations and a softmax output. Training uses a deterministic seed and a fixed validation split, and the current model reaches:

validation accuracy: 1.00

training accuracy: 1.00

samples: 294

Evidence: `training/models/training_metrics.json`

Portable export and deployment

The training script saves the trained weights as a portable NumPy payload, and the PTQ export step creates a portable model payload for the inference service. This keeps the assignment runnable without TensorFlow while still documenting the intended full INT8 deployment target.

The deployment playbook orchestrates the runtime service and model copy path for the truck fleet with a local inference container and local MQTT environment.

Inference and monitoring

The inference service loads the normalised features, runs the model, and writes a result to the local alert log. It optionally publishes the alert over MQTT with QoS 1 semantics. The monitoring component computes PSI on confidence scores to flag model drift.

Benchmark results

The benchmark compares three deployment variants: FP32, PTQ INT8, and pruned PTQ INT8.

The pruned PTQ variant is the best engineering trade-off: it retains near-full accuracy while reducing model size and latency, which directly supports the fleet’s edge deployment constraints.

Conclusion

LogiEdge meets the assignment goal of a practical offline cold-chain edge AI solution. It keeps inference local, reduces telemetry cost, preserves safety during network loss, and demonstrates a measurable performance trade-off between accuracy and edge efficiency. The current implementation is deterministic, reproducible, and aligned with the project’s architecture and benchmarking evidence.

Benchmark results

Variant	Mean latency (ms)	P95 latency (ms)	Model size (KB)	Accuracy (%)	Energy (mJ)
M1_FP32	0.0313	0.0340	8.4	100.0	0.2351
M2_PTQ_INT8	0.0117	0.0118	2.8	100.0	0.0879
M3_PRUNED_PTQ_INT8	0.0106	0.0154	1.9	99.4	0.0796